

PROJECT REPORT

Project Code: 24CC3046-P068

AZURE APPLICATION INSIGHTS INSTRUMENTATION FOR A WEB APP

Department: Computer Science and Engineering (CSE)

College: KL University

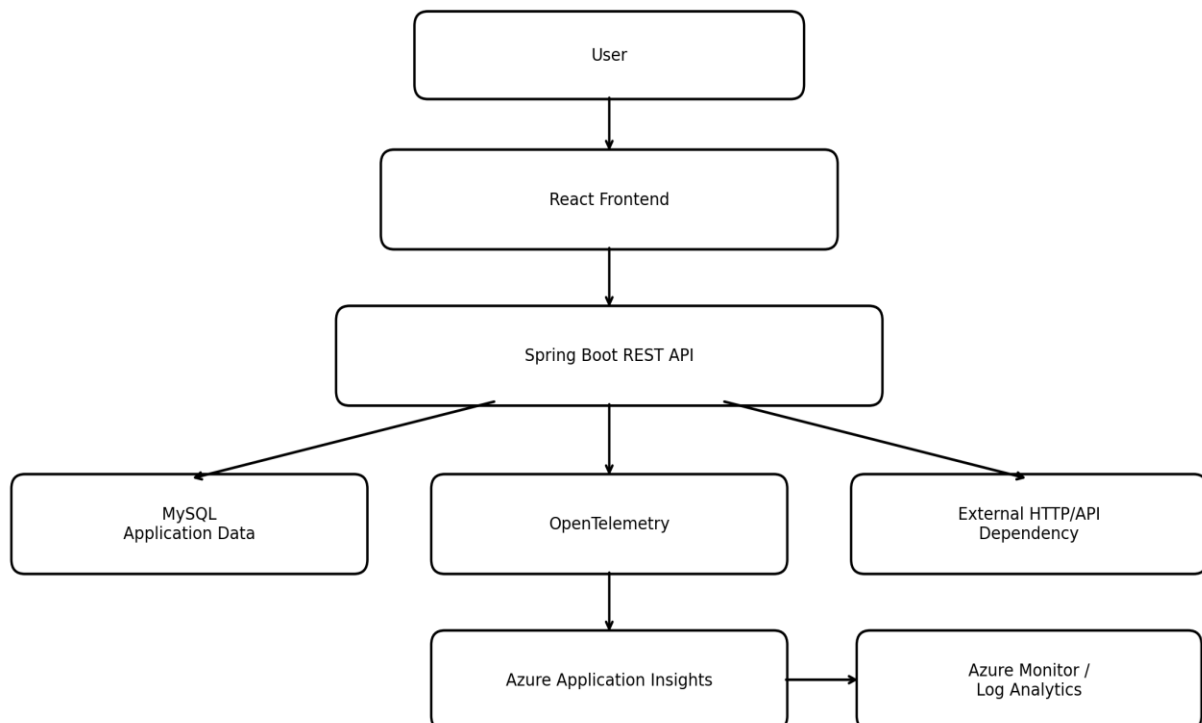
Academic Year: 2026–2027

TEAM : T272

TEAM MEMBERS

STUDENT ID	STUDENT NAME
2400030235	B Tarun
2400030268	A Srinithi
2400030570	Aaftaab Rabbani Khan

Proposed System Architecture



Abstract

This project focuses on **monitoring and analyzing a web application using Azure Application Insights and OpenTelemetry**. The application uses a Spring Boot backend, MySQL database, REST APIs, and secure authentication. Application Insights collects information about requests, dependencies, exceptions, and performance, enabling end-to-end tracing and identification of slow or failed components. The project demonstrates how cloud-based observability can help developers **detect errors, identify performance bottlenecks, and improve application reliability**.

Project Overview

This project develops a small web application and instruments it with Azure Application Insights and OpenTelemetry. The goal is to observe application requests, dependencies, failures, performance and end-to-end transactions so that bottlenecks and errors can be identified quickly.

The application itself will contain normal business data and APIs. Azure Application Insights will handle monitoring telemetry rather than replacing the application database.

Problem Statement

Traditional console logs can show that something went wrong, but they do not always make it easy to determine which request, database call or external dependency caused the delay or failure. The project addresses this observability problem by correlating application telemetry in Azure.

Aim

To implement and monitor a web application using Azure Application Insights and OpenTelemetry, with emphasis on end-to-end request tracing, dependency mapping, performance analysis and failure detection.

Objectives

- Build a functional web application with a frontend, backend REST APIs and MySQL database.
- Implement secure authentication and role-based authorization using Spring Security and JWT.
- Instrument the backend with OpenTelemetry and connect telemetry to Azure Application Insights.
- Monitor requests, dependencies, traces, exceptions and performance.

- Identify the slowest dependency and demonstrate a controlled failure scenario.
- Introduce basic KQL for querying telemetry after the monitoring pipeline is working.
- Configure useful alerts and prepare a final monitoring demonstration.

Use Cases

• Trace Requests End-to-End with Dependency Mapping

This use case tracks a user's request from the beginning to the end of its execution. Application Insights shows the request and the different dependencies involved, such as the database or external APIs. This helps developers understand the complete request flow and quickly identify where a failure or delay occurred.

• Find the Slowest Dependency

This use case identifies which dependency is taking the most time during a request. Application Insights records the execution time of database calls, HTTP services, and other dependencies, allowing developers to locate the main performance bottleneck.

Proposed Technology Stack

Layer	Technology	Purpose
Frontend	React.js	User interface and API interaction
Backend	Java + Spring Boot	REST services and business logic
Database	MySQL	Application/business data
Persistence	Spring Data JPA + Hibernate	Database access
Security	Spring Security + JWT	Authentication and authorization
Monitoring	Azure Application Insights	Application performance monitoring
Telemetry	OpenTelemetry	Instrumentation and telemetry collection
Analysis	Azure Monitor / Log Analytics / KQL	Telemetry analysis
Testing	Postman	API testing
Version Control	Git + GitHub	Source-code management

Application Design

The recommended application is a simple Order Management Web Application. It provides enough real API and database activity to demonstrate monitoring clearly.

- User registration and login
- Product listing and product management
- Create, view, update and delete/cancel orders
- User and administrator roles
- Database-backed REST APIs

Database Design

MySQL stores business/application data. A simple design is:

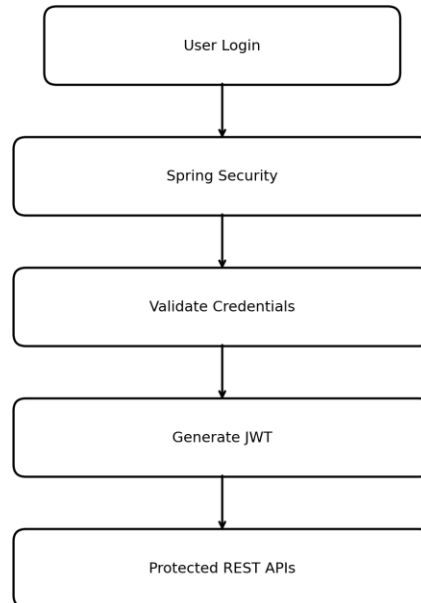
Table	Important fields
users	id, name, email, password_hash, role
products	id, name, price, stock
orders	id, user_id, total, status, created_at
order_items	id, order_id, product_id, quantity, price

REST API Layer

Area	Example endpoints
Authentication	POST /api/auth/register POST /api/auth/login
Products	GET /api/products POST /api/products PUT /api/products/{id} DELETE /api/products/{id}
Orders	POST /api/orders GET /api/orders GET /api/orders/{id} PUT /api/orders/{id} DELETE /api/orders/{id}

Authentication & Authorization

Authentication and Authorization Flow



- Spring Security validates the username and password during login.
- A JWT is generated after successful authentication.
- The frontend sends the JWT with protected API requests.
- Spring Security validates the JWT before allowing access.
- Roles such as USER and ADMIN can control which APIs are accessible.
- Passwords, JWTs and other secrets must not be written to application logs or telemetry.

Azure Application Insights

Azure Monitor Application Insights is the application-performance-monitoring component used in this project. For supported scenarios, OpenTelemetry can instrument the application and send telemetry for analysis in Application Insights.

- Requests: HTTP requests and their duration/success.
- Dependencies: calls to databases, HTTP services and other supported dependencies.
- Exceptions: application errors and stack traces.
- Traces/logs: diagnostic information emitted by the application.
- Metrics: performance and runtime measurements supported by the instrumentation.
- Application Map and transaction diagnostics: visual and end-to-end investigation of dependencies and requests.

OpenTelemetry

OpenTelemetry is the instrumentation layer. It provides a standardized way to collect telemetry such as traces, metrics and logs. The Azure Monitor OpenTelemetry distribution connects this telemetry to Azure Monitor/Application Insights.

Basic pipeline:

Spring Boot → OpenTelemetry → Azure Monitor Export → Application Insights → Azure Monitor / Logs

End-to-End Tracing & Dependency Mapping

A single order request may call authentication logic, the MySQL database and an external HTTP service. The monitoring layer should allow the request and related dependency operations to be correlated.

Example:

POST /api/orders → Authentication → MySQL → External Notification API → Response

This directly supports the project's use cases: tracing requests end-to-end and finding the slowest dependency.

Sampling & Instrumentation Overhead

- Sampling reduces telemetry volume, cost and processing load when an application produces large amounts of telemetry.
- Sampling must be configured carefully because excessive sampling can reduce the amount of telemetry available for diagnosis.
- Instrumentation also introduces some application overhead, so telemetry should be useful and appropriately configured.

Alerts & Dashboard

- Create useful alerts for high failure rates or abnormal response times.
- Use dashboards to summarize request count, failures, response time and dependency performance.
- Use Application Map and transaction diagnostics during troubleshooting.

Advantages

- End-to-end visibility into application behavior.
- Faster identification of performance bottlenecks.

- Correlation of requests, dependencies and exceptions.
- Centralized cloud-based monitoring.
- Useful foundation for production observability.

Limitations

- Telemetry collection can add some overhead.
- Sampling can hide some low-frequency telemetry when configured aggressively.
- Cloud monitoring can create ingestion/storage costs.
- Sensitive data must be excluded from logs and telemetry.

Final Demonstration Flow

1. Open the web application → 2. Login → 3. Create/view an order → 4. Show the REST request → 5. Open Application Insights → 6. Show Application Map → 7. Show request performance → 8. Open transaction/dependency details → 9. Demonstrate a slow or failed dependency → 10. Run a basic KQL query → 11. Show the alert/dashboard.

Future Enhancements

- Deploy the application to Azure App Service or another Azure hosting option.
- Add more external services/microservices to demonstrate distributed tracing.
- Add custom metrics and events.
- Create richer dashboards/workbooks.
- Add availability monitoring and more advanced alerts.
- Expand KQL-based analytics.

Conclusion

The project combines a practical web application with modern cloud observability. The application handles users, products and orders, while Azure Application Insights and OpenTelemetry provide visibility into requests, dependencies, traces, exceptions and performance. The project will demonstrate not only how to monitor an application, but also how monitoring can be used to locate a real bottleneck or failure.